

ASK FOR A SETUP SCREEN

AI will build you a classroom tool in about four minutes. **Then it will tell you to open the file in a text editor and change the part between the brackets**, and that is the step where almost everybody quietly stops using it.

One sentence, added to whatever you were going to ask for, prevents it.

"Put a setup screen inside the file so I can change the content without editing any code." That is the whole request, and it is the difference between a tool you adapt for four years and a demo you never open again.

Why editing the code is the step people give up at

It is not that it is hard. It is that it is frightening in a way that is completely rational: there is no undo, no save button you trust, and no way to tell whether the thing you deleted mattered.

So it gets done once, carefully, in August — **and then never again**, which means the tool is frozen on the day you made it. A tool you cannot change is a worksheet with extra steps.

Where this comes from

I teach high school woodshop and build the software my classes use. **I shipped a classroom game that asked teachers to edit a block of code**, watched what that actually meant for somebody who does not write code for a living, and rebuilt it the same evening around a setup screen. **The tool did not change. Who could use it changed completely.**

Inside, in five pages

What a setup screen is and **the three questions that decide what goes on it** · the request written out, and what to do when you get a code block anyway · **the paste-and-fill shortcut** that gets AI to draft your content in about thirty seconds · the one parser rule that stops it failing silently · and a fill-in page.

THE IDEA

What a setup screen is, and what belongs on it

A screen inside the tool, reached from a button, where **everything that is yours rather than the tool's** gets typed in. Your questions. Your class list. Your prices. Your room. Nothing else.

Three questions decide what goes on it

- 1 • **Would a different teacher need to change this?** Your questions, yes. The color of the buttons, no. **If everybody would change it, it belongs on the screen.**
- 2 • **Would *you* change it again next term?** A class list changes every year and a price list changes every month. Those belong on the screen even if nobody else would touch them.
- 3 • **Does getting it wrong break the tool?** If the answer is yes, it belongs on the screen **with a check beside it** — not in the code where a stray character ends the lesson.

And the test that settles any argument

Hand the file to a colleague who has never seen it and say nothing. If they can make it theirs, the screen is right. If they ask you a question, the answer to that question belongs on the screen — which is the same rule as user testing, and the same reason it works.

What should not go on it

Everything else. A setup screen with thirty options is a settings menu, and a settings menu is a thing people close. **Six to ten fields is a tool. Thirty is a chore.** If you cannot decide, leave it out — it is far easier to add a field later than to take one away from somebody who started using it.

The reason this is the most useful thing on this page.

A tool you can change is a tool you still use in March. **Everything else about building with AI is downstream of whether the thing survives contact with a second week.**

Written out, and what to do when it ignores you

Add this to whatever you were going to ask for. It works on a new build and it works on something you already have.

ADD TO ANY BUILD REQUEST

Put a **setup screen inside the file**, reached from a button, where I can change the content without editing any code or opening a text editor.

On it: **<list the things that are yours – questions, names, prices>**

Rules:

- It saves what I type and still has it when I reopen the file.
- It tells me when something I typed will not work, and says why.
- Everything else stays where it is. Do not redesign the rest.

WHEN IT HANDS YOU A CODE BLOCK ANYWAY

That still asks me to edit code. I need a screen with input boxes, inside the running page, that a person who has never seen code could use. Keep everything else exactly as it is.

The three things it will get wrong, and they are the same three every time

It forgets. You type your questions in, close the file, reopen it, and they are gone. Ask for it to still be there after a reload, in those words.

It redesigns everything else. You asked for a screen and got a new color scheme. Say "do not change anything I did not ask about" and say it again if you have to.

It accepts anything. A blank field, a stray comma, a duplicate — all swallowed, and the tool quietly misbehaves later. The next page is about that one, because it is the one that costs you a lesson.

Say what should be true, not what to build.

"A screen a person who has never seen code could use" is a requirement you can check. "A nice settings page" is a wish. **The first one gets tested; the second gets agreed with.**

Let AI fill the screen, and make the parser honest

A setup screen you have to type into by hand on a Sunday is still work. **The second half of the request is a paste box**, and a prompt that fills it in about thirty seconds.

ADD THIS TO THE SAME REQUEST

Also give me a **paste box** on the setup screen that accepts a plain list, one item per line, and a **copy button for a prompt** I can paste into any AI chat tool to produce that list in the right format.

Why one item per line beats anything cleverer

It is the format a spreadsheet column already gives you, it is the format an AI returns without being argued with, and **it is the only format you can check by looking at it**. If you need more than one piece of information per line, separate them with a vertical bar rather than a comma — **commas are already in your content** and a comma format breaks the first time somebody writes "Smith, Alex".

The one parser rule, and it is the whole page

Ask for this in those words: "when a line cannot be used, tell me which line and why, and keep the rest."

The default behavior is to silently skip anything it does not understand. **You paste thirty questions, twenty-eight load, and nothing tells you** — and you find out in front of a class. A parser that names what it dropped turns a lost lesson into four seconds of reading.

And the instruction that always goes with the drafting prompt

Read what it gives you before you use it. AI will write you twenty plausible review questions in thirty seconds and two of them will be wrong, or off your standard, or ask something you have not taught yet. **Thirty seconds of drafting still needs two minutes of reading**, and you own the standards either way.

Design the setup screen before you ask for anything

A tool you want, or one you have and never changed. Fill this in first, then paste it into the request on page 3.

The tool:

What goes on the screen

Why: everybody changes it, I change it yearly, or it breaks things

What I left off, and why

Six to ten fields is a tool, thirty is a chore. Name one thing you were tempted by and cut.

The paste box: one item per line, and each line is

For example: question | answer | wrong | wrong, or just a name. **Vertical bars, never commas.**

The test when it comes back, and where to start instead

Hand it to somebody who has never seen it and say nothing. **If they ask you a question, the answer belongs on the screen.**

Or start from one that works: our **Vibe Coding for Teachers** tools each ship as a working file with the setup screen already in it and no code to edit. Search our store.

If this was useful, would you leave a review?

TPT gives you credit toward future purchases, and reviews decide whether another teacher sees this.